Overview

execution map — The map aggregates request handling into parsing followed by validation and response rendering.

The execution map [1] first shows the ownership path. The following stages unpack its five reader decisions: coordinate, normalize, validate, then render one of two outcomes.

Mental model

- Question: How does a raw request become a success or error response?
- State: N/A — The overview introduces the runtime model.

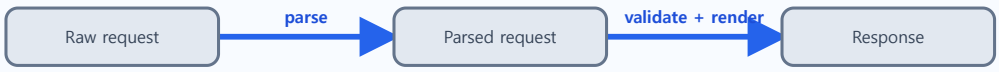
Responsibilities

app.handle

- **Does:** Coordinate parsing, validation, and response rendering.

1. Coordinate the request lifecycle

Execution position Current explanation scope: parse · validate + render



The lifecycle coordinator is 1. The overview map remains the high-level ownership reference, so this stage can focus on the source-level branch.

Mental model

- Question: Where is the complete request process coordinated?
- Trigger: app.handle receives raw_request.

State changes

request context

- **Owner:** app.handle
- **Before:** raw_request only
- **Change:** build_context, parsing, and validation run
- **After:** parsed request plus validation result
- **Must remain true:** response rendering receives either parsed data or a problem.

Responsibilities

app.handle

- **Does:** Sequence collaborators and choose the success or error branch.

Failure and alternate outcomes

- validation returns false → render_error receives the validation problem.

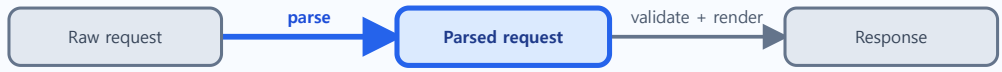
[1] Source **src/app.lua** Function: app.handle L13–L23

app.handle creates the parsed request, branches on validation, and selects the response renderer.

```lua
    raw_request = raw_request,
    received_at = "demo-clock",
  }
end

function app.handle(raw_request)
  local context = build_context(raw_request)
  local parsed = request.parse_request(context.raw_request)
  local ok, problem = request.validate_request(parsed)

  if not ok then
    return response.render_error(problem)
  end

  return response.render_response(parsed)
end

return app
```

2. Normalize optional request data

Execution position Current explanation scope: Parsed request · parse



The normalization contract is implemented in 1.

Mental model

- Question: How do optional raw fields become a predictable request value?
- Trigger: app.handle passes context.raw_request to request.parse_request.

State changes

request

- **Owner:** request.parse_request
- **Before:** method, path, and user may be absent
- **Change:** fallback values are applied
- **After:** decoded table contains all three fields
- **Must remain true:** later stages can read method, path, and user without nil checks.

Responsibilities

request.parse_request

- **Does:** Define the normalized request contract.
- Failure: N/A — Missing optional fields receive defaults instead of failing parsing.

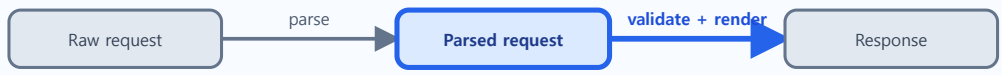
[1] Source `src/request.lua` Function: request.parse_request L3-L13

parse_request supplies defaults and returns the normalized table.

```lua
local request = {}

function request.parse_request(raw_request)
  local method = raw_request.method or "GET"
  local path = raw_request.path or "/"
  local user = raw_request.user or "anonymous"

  return {
    method = method,
    path = path,
    user = user,
  }
end

function request.validate_request(parsed)
  if parsed.method ~= "GET" and parsed.method ~= "POST" then
    return false, "unsupported method"
  end
```

3. Reject unsupported or incomplete requests

Execution position Current explanation scope: Parsed request · validate + render



The guard conditions and failure result are 1.

Mental model

- Question: Which conditions decide whether rendering may continue?
- Trigger: request.validate_request receives the normalized request.

State changes

validation result

- **Owner:** request.validate_request
- **Before:** decoded request is unclassified
- **Change:** method and path guards execute
- **After:** true or false with an optional problem
- **Must remain true:** false always carries a human-readable reason.

Responsibilities

request.validate_request

- **Does:** Enforce supported methods and non-empty paths.

Failure and alternate outcomes

- method is unsupported or path is empty → return false and a problem for app.handle.

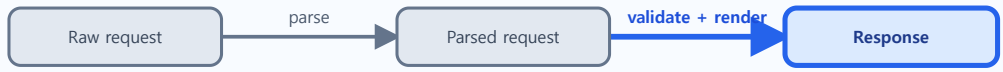
[1] Source **src/request.lua** Function: request.validate_request L15–L25

validate_request turns boundary violations into a boolean and problem pair.

```
    path = path,
    user = user,
  }
end

function request.validate_request(parsed)
  if parsed.method ~= "GET" and parsed.method ~= "POST" then
    return false, "unsupported method"
  end

  if parsed.path == "" then
    return false, "missing path"
  end

  return true, nil
end

return request

```

4. Render the valid response

Execution position Current explanation scope: Response · validate + render



The success response construction is 1.

Mental model

- Question: How is a validated request exposed as a successful response?
- Trigger: app.handle receives ok equal to true.

State changes

response

- **Owner:** response.render_response
- **Before:** parsed request is validated but unrendered
- **Change:** render_response formats status and body
- **After:** HTTP 200 greeting response
- **Must remain true:** status formatting is delegated to status_line.

Responsibilities

response.render_response

- **Does:** Build the success response from normalized request data.
- Failure: N/A — Validation has already selected the success branch.

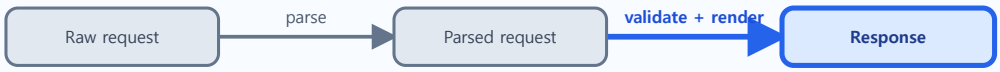
[1] Source **src/response.lua** Function: response.render_response L7–L12

render_response builds a shared-format 200 response from parsed fields.

```lua

local function status_line(code)
  return "HTTP/1.1 " .. tostring(code)
end

function response.render_response(parsed)
  return {
    status = status_line(200),
    body = "Hello, " .. parsed.user .. " from " .. parsed.path,
  }
end

function response.render_error(problem)
  return {
    status = status_line(400),
    body = "Bad request: " .. problem,
```

5. Render the validation error

Execution position Current explanation scope: Response · validate + render



The alternate response construction is 1.

Mental model

- Question: How does a failed validation become an observable error response?
- Trigger: app.handle receives ok equal to false and a problem.

State changes

response

- **Owner:** response.render_error
- **Before:** validation problem has no transport representation
- **Change:** render_error formats status and body
- **After:** HTTP 400 error response
- **Must remain true:** the original problem remains visible in the body.

Responsibilities

response.render_error

- **Does:** Translate the validation problem into the error response shape.
- Failure: N/A — This stage is the selected alternate outcome.

[1] Source **src/response.lua** Function: response.render_error L14–L19

render_error preserves the problem while producing the shared-format 400 response.

```lua
      status = status_line(200),
      body = "Hello, " .. parsed.user .. " from " .. parsed.path,
    }
end

function response.render_error(problem)
  return {
    status = status_line(400),
    body = "Bad request: " .. problem,
  }
end

return response

```